A Geometric and Computational Investigation of the Lonely Runner Conjecture

Asfi Hassan
Department of Computer Science
University of Windsor
Windsor, Ontario, Canada

October 27, 2026

Abstract

This report covers the first part of my COMP-4960 project on the Lonely Runner Conjecture, which continues my Summer 2026 USRA research with Dr. Ahmad Biniaz. We study speed sets with $v_1 = 1$ and $v_n \leq 2n$, the open part of a question raised by Tao. So far this term, I rewrote the summer programs so that they use whole numbers instead of fractions and check every speed set at once. The new program gives the same answers as the summer code and is about 660 times faster. This let us extend the summer verification from $n \leq 10$ to $n \leq 14$, about 27 million speed sets in total. We also found and corrected a mistake in the summer survey about the pair of runners that realises the best lonely time, and answered the question the survey left open. The report ends with a plan for the rest of the year.

1 Introduction

COMP-4960 is a two-term undergraduate research course offered by the School of Computer Science at the University of Windsor. The course provides students with an opportunity to conduct independent research under the guidance of a faculty supervisor, explore research problems, and develop practical experience in academic research. For this course, I will be continuing my previous research under the supervision of Dr. Ahmad Biniaz, focusing on the Lonely Runner Conjecture through geometric and computational approaches.

This project builds upon the research I conducted during Summer 2026 through the Natural Sciences and Engineering Research Council of Canada (NSERC) Undergraduate Student Research Award (USRA), with additional funding provided through Dr. Biniaz's research grant. During this research term, I had the opportunity to study an open mathematical problem, review existing literature, explore different proof techniques, and develop software to investigate the problem computationally. Continuing this work through COMP-4960 will allow me to build on the progress made during the summer and investigate questions that remain unresolved.

The Lonely Runner Conjecture is a well-known open problem in number theory and combinatorics. It considers a group of runners moving around a circular track at distinct constant speeds, all starting from the same position. The conjecture states that, for any group of $n+1$ runners, each runner will eventually be at least $1/(n+1)$ of the track's circumference away from every other runner. Although the problem is relatively simple to describe, proving that this condition holds for every possible configuration of speeds has remained a significant mathematical challenge.

Our research focuses on studying this problem from a computational geometry perspective. By fixing one runner as stationary, the distances of the remaining runners can be represented as piecewise-linear functions over time. This geometric interpretation allows us to examine intersections, identify potential lonely times, and investigate patterns that may help explain why the conjecture holds for particular speed configurations.

During the summer research term, I developed Python-based visualization and exhaustive search programs to study these geometric properties. Our investigation concentrated on speed sets where the smallest speed is $v_1 = 1$ and the largest speed satisfies $v_n \leq 2n$, a special case connected to an open question raised by Terence Tao. Through computational experiments, we verified previously established cases and tested several possible geometric properties. While some of our initial hypotheses were disproven by counterexamples, these findings helped us better understand the limitations of certain approaches and identify potential directions for further investigation.

For the current project course, the intention is to continue this research by refining the existing computational tools, exploring additional geometric properties, and investigating whether new patterns or structural relationships can be identified. Particular attention will be given to improving the efficiency of the search algorithms and extending the computational experiments to larger speed sets. The overall objective is to build upon the summer research findings and contribute to a better understanding of the geometric structure underlying the Lonely Runner Conjecture.

2 Background

2.1 The problem in our notation

We keep the conventions of the summer survey. The number n counts the *moving* runners, so there are $n+1$ runners in total and the target distance is $1/(n+1)$. One runner stays at the start, and the others have distinct positive whole number speeds $V = \{v_1 < v_2 < \cdots < v_n\}$.

For a number x, we write $\|x\|$ for the distance from x to the nearest whole number, so $\|2.3\| = 0.3$ and $\|2.8\| = 0.2$. At time t, the runner with speed v is at distance $\|tv\|$ from the start (measured around the track). A time t is a *lonely time* for V if every runner is far enough away:

$$\|tv\| \geq \frac{1}{n+1} \quad \text{for every } v \in V.$$

The best possible distance is

$$\kappa(V) = \max_{0<t<1} \min_{v \in V} \|tv\|,$$

so the conjecture says $\kappa(V) \geq 1/(n+1)$ for every speed set. A speed set is called *tight* if $\kappa(V) = 1/(n+1)$ exactly. The simplest example is $\{1, 2, \ldots, n\}$.

2.2 The geometric picture

If we plot $\|tv\|$ against t, we get a sawtooth made of v triangles of height $1/2$. Drawing all the runners together, the smallest value at each time forms the *lower envelope*, and $\kappa(V)$ is its highest point. Figure 1 shows an example with $n = 8$.

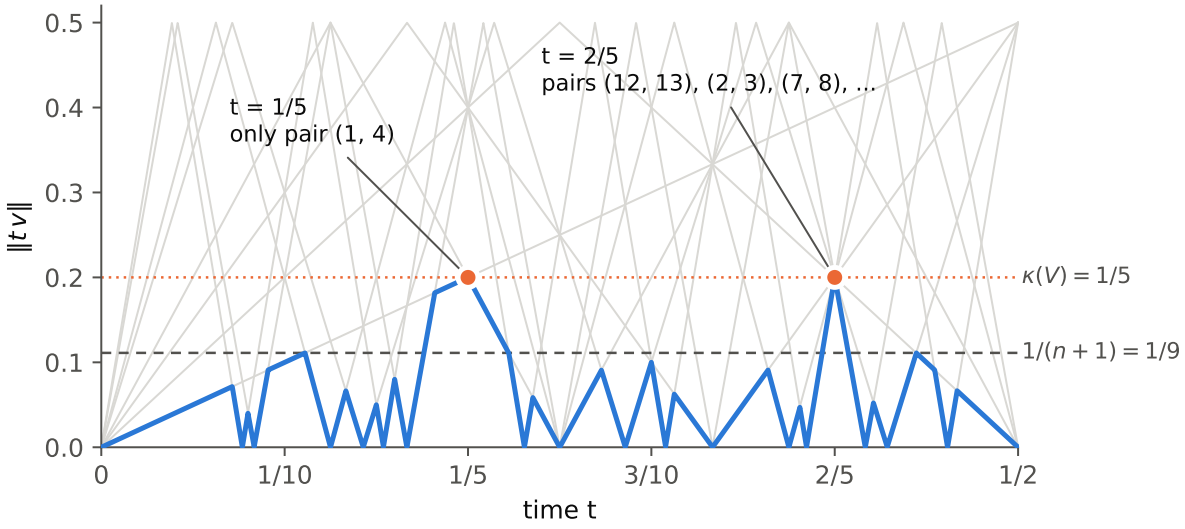


Figure 1: The speed set $V = \{1, 2, 3, 4, 7, 8, 12, 13\}$, $n = 8$. Grey lines are the eight sawtooths and the blue line is the lower envelope. Its highest value is $\kappa(V) = 1/5$, reached twice in $(0, 1/2]$: at $t = 1/5$ and at $t = 2/5$. This example comes back in Section 4.3.

2.3 What we know from the summer

Two facts from the summer survey are used throughout this report.

Fact 1. *If $v_1 \geq 2$ and $v_n \leq 2n$, then the speed set always has a lonely time. So for Tao's question only the case $v_1 = 1$ is open.*

Fact 2. *The highest point of the envelope is always at a time of the form $t = a/(v_i + v_j)$, where $v_i, v_j \in V$ and $i = j$ is allowed. In particular, every time we need to check is a fraction a/m with $m \leq 2v_n \leq 4n$.*

Fact 2 is what makes a computer search possible: instead of checking infinitely many times, we only check a finite list of fractions.

In the summer, we used these facts to verify the conjecture for all $125{,}476$ speed sets with $v_1 = 1$, $v_n \leq 2n$ and $n \leq 10$. We also tested two geometric hypotheses (both false) and seven possible rules for which pair of runners realises $\kappa(V)$. The survey reported that the last of these rules, $\max(v_i, v_j) > n/2$, first fails at $n = 8$, and asked whether the weaker rule $\max(v_i, v_j) \geq n/2$ survives for $n \geq 9$.

What is known. The full conjecture has been proved for up to 13 runners ($n \leq 12$) [1, 4, 2], and a September 2026 preprint claims 14 and 15 runners ($n \leq 14$) [26]. For bounded speeds, Tao proved the conjecture when every speed is at most $1.2n$ and suggested $2n$ as the natural next target [6]. Bohman and Peng proved it for large n when the speeds are a little below $2n$ [7].

3 Methods

The summer programs worked correctly but were slow. To check $n = 10$, the verification program needed about four minutes, and each step up in n multiplies the number of speed sets by almost 4. This term I rewrote the core of the programs using two simple ideas.

3.1 Idea 1: avoid fractions

The summer programs used Python's `Fraction` type. It is exact, which matters because tight sets sit exactly on the bound, but it is slow. Fact 2 says we only need times of the form $t = a/m$, and at such a time every distance is a whole number divided by m:

$$\left\| \tfrac{av}{m} \right\| = \frac{d}{m}, \qquad \text{where } r = (av) \bmod m \text{ and } d = \min(r, m - r).$$

So we can store only the whole number d. To compare two distances d_1/m_1 and d_2/m_2 we compare $d_1 m_2$ with $d_2 m_1$, which is still exact.

Example. Take $V = \{1, 3, 4\}$ ($n = 3$, bound $1/4$) and $t = 3/7$. The remainders $3v \bmod 7$ are 3, 2 and 5, so the distances are $3/7$, $2/7$ and $2/7$. The smallest is $2/7$, and since $2 \cdot 4 = 8 \geq 7$ we have $2/7 \geq 1/4$, so $t = 3/7$ is a lonely time. No fractions were needed.

3.2 Idea 2: check all speed sets at once

The summer programs looped over the speed sets, and for each set looped over the times. We swapped the two loops: for each time, we handle all the speed sets together.

Each speed set is stored as a *bitmask*, a whole number whose bit $v-1$ is 1 exactly when v is in the set. For example, $\{1,3,4\}$ is stored as binary 1101, which is 13. Now fix one time $t = a/m$. Every possible speed $v \le 2n$ has its own distance d_v/m at this time, so we group the speeds by distance: L_1 holds the speeds at distance $1/m$, L_2 the speeds at distance $2/m$, and so on. The distance of a whole speed set V at time t is then the first d for which V and L_d share a speed, which a computer checks with one bitwise AND.

Example. With $n = 3$ and $t = 3/7$, the speeds $1,\dots,6$ have $d = 3,1,2,2,1,3$. So $L_1 = \{2,5\}$, $L_2 = \{3,4\}$ and $L_3 = \{1,6\}$. The set $V = \{1,3,4\}$ shares nothing with L_1 but shares 3 and 4 with L_2, so its distance is $2/7$, as before.

Using the `numpy` library, the bitwise operations are applied to the whole array of speed sets in one step. The new program, `lrc_batch.py`, loops over the times that Fact 2 allows (only 245 of them for $n = 10$), keeps the best distance found so far for every set, and finally records every time and pair of runners that reaches $\kappa(V)$, using the same definition as the summer program `patterns.py`.

3.3 Making sure the new programs are right

A faster program is only useful if it gives the right answers, so we compared it with the summer code and with known results.

- For all $8{,}788$ speed sets with $n \le 8$, the new code gives exactly the same $\kappa(V)$ and exactly the same list of realising pairs as the summer program `patterns.py`.
- The batch program agrees with a simple one set at a time version on every set with $n \le 8$, and on $2{,}000$ randomly chosen sets for each of $n = 9, 10, 11$.
- The tight sets it finds match the ones found by Goddyn and Wong [25] (Section 4.2).

4 Results

4.1 Speed

Table 1 and Figure 2 compare the two programs on the same computer. At $n = 10$ the new program takes 0.39 seconds instead of about 259, roughly 660 times faster. It handles $n = 13$ (over five million speed sets) in just over a minute.

n	speed sets	summer program	new batch program
8	6,435	12.1 s	0.02 s
9	24,310	60.5 s	0.07 s
10	92,378	258.9 s	0.39 s
11	352,716		1.9 s
12	1,352,078		10.0 s
13	5,200,300		73.5 s
14	20,058,300		about 8 min

Table 1: Time to check every speed set with $v_1 = 1$ and $v_n \le 2n$ for one value of n. The summer program was not run past $n = 10$.

4.2 Verification up to $n = 14$

With the new program we checked every speed set with $v_1 = 1$ and $v_n \le 2n$ for $n = 11$, 12, 13 and 14. Every one of them has a lonely time. Together with the summer results, this covers all $27{,}088{,}870$ speed sets with $n \le 14$ (Table 2). By Fact 1, the answer to Tao's question is therefore yes for every $n \le 14$.

n	speed sets checked	sets with no lonely time
1 to 10 (summer)	125,476	0
11	352,716	0
12	1,352,078	0
13	5,200,300	0
14	20,058,300	0
total	27,088,870	0

Table 2: Verification of the conjecture for $v_1 = 1$ and $v_n \le 2n$.

This agrees with what is already known, since the full conjecture is proved for $n \le 12$ and claimed for $n \le 14$. So it is a strong test of our program rather than a new result. It also shows the limit of checking sets one by one: $n = 15$ would have about 78 million sets and $n = 20$ almost 7×10^{10}. Going further needs a different idea, which is part of the plan in Section 5.

Tight sets. The same run lists every tight speed set in the range. For $n \le 14$, apart from $\{1,2,\ldots,n\}$, there are only five:

$$\{1,3,4,7\}, \qquad \{1,3,4,5,9\}, \qquad \{1,2,3,4,5,7,12\},$$
$$\{1,4,5,6,7,11,13\}, \qquad \{1,2,\ldots,11,13,24\}.$$

All five were already found by Goddyn and Wong [25]. Two of them follow a pattern they describe: take $\{1,\ldots,n\}$ and replace one speed r by $2r$ (here $6 \to 12$ for $n = 7$, and $12 \to 24$ for $n = 13$).

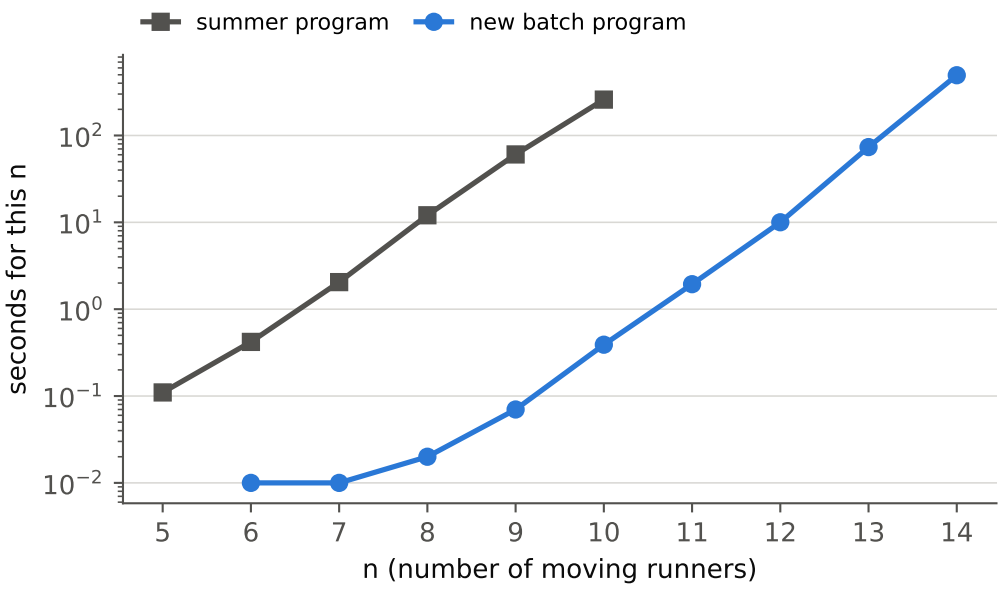


Figure 2: Seconds needed for each n, on a log scale. Both lines grow at about the same rate, because the number of speed sets grows by almost 4 each step, but the new program is hundreds of times faster at every n.

4.3 A correction to the summer survey

The summer survey said that the rule $\max(v_i, v_j) > n/2$ first fails at $n = 8$, on $V = \{1,2,3,4,7,8,12,13\}$, where $\kappa(V) = 1/5$ was said to be reached only by the pair $(1,4)$. This is not correct. Figure 1 shows that the envelope reaches $1/5$ twice: at $t = 1/5$ through the pair $(1,4)$, and also at $t = 2/5$, where speeds $3,8,13$ are rising and speeds $2,7,12$ are falling at height $1/5$. The pair $(12,13)$ realises $\kappa(V)$ at $t = 2/5$, and $13 > n/2$, so the rule holds for this set.

Running the summer program `patterns.py` unchanged on all $6{,}435$ sets with $n = 8$ also reports no failure. So the program was right, and the mistake was in the write up, which most likely looked only at the first time where the maximum is reached.

4.4 The size of the realising pair

With the faster program we repeated this test up to $n = 13$, for both the original rule and the weaker rule $\max(v_i, v_j) \geq n/2$. As in the summer, a rule counts as satisfied when *at least one* realising pair meets it. Table 3 shows the results.

So the rule $\max(v_i, v_j) > n/2$ actually holds for every $n \leq 9$ and first fails at $n = 10$, on

$$V = \{1,2,3,4,5,8,9,10,14,15\}.$$

The weaker rule $\max(v_i, v_j) \geq n/2$ holds for every $n \leq 10$ and first fails at $n = 11$, on the same set with 16 added. This answers the question left open in the summer survey: the weaker rule does not survive either.

		sets where the rule fails			hard sets
n	speed sets	$\max > n/2$	$\max \geq n/2$	hard sets	where it fails
8	6,435	0	0	602	0
9	24,310	0	0	2,868	0
10	92,378	50	0	6,953	0
11	352,716	218	218	25,971	0
12	1,352,078	839	76	90,606	0
13	5,200,300	1,152	1,152	297,986	0

Table 3: Speed sets where no realising pair satisfies the rule. For $n \leq 7$ there are no failures. Hard sets are defined below.

Why it fails. These sets contain no multiple of 6. At $t = 1/6$ a runner can only be at distance 0, $1/6$, $2/6$ or $3/6$, and distance 0 would need a multiple of 6. So every runner is at distance at least $1/6$, far more than $1/(n+1)$. The best time is $t = 1/6$, realised only by the slow pair $(1,5)$, which is why the rule fails.

The same reasoning works for any $m \leq n+1$: if V contains no multiple of m, then $t = 1/m$ is a lonely time. We call a speed set *hard* if this shortcut never works, that is, if V contains a multiple of every m from 2 to $n+1$. These are the sets where the conjecture is actually difficult. The last two columns of Table 3 show that among hard sets the rule never failed for $n \leq 13$. This gives a cleaner question for the next stage:

> **Question.** If V is hard, is $\kappa(V)$ always realised by a pair containing a speed larger than $n/2$?

The program also saves all of this data as CSV files (one row per speed set for $n \leq 10$, with every best time, every realising pair, and their sums and greatest common divisors), which was another item on the summer's further work list.

5 Conclusions and plan

In the first part of the fall term we made the summer programs exact without fractions and hundreds of times faster, checked them carefully against the summer code, and used them to extend the verification of Tao's question from $n \leq 10$ to $n \leq 14$. We also corrected one claim from the summer survey, showed that the size rule for the realising pair fails at $n = 10$ and its weaker version at $n = 11$, and found that every failure comes from an easy set. That led to a sharper question about hard sets.

Plan for the rest of the project

November and December 2026 (COMP-4960A).

1. **A faster way to verify.** Checking sets one by one will not get far past $n = 14$. Instead of starting from the speed sets, we will start from the times: for each time t, list the speeds that are too close to the start at t. A set with no lonely time would

have to contain at least one speed from every such list, and we will try to show with a search that this is impossible, without listing the sets. A recent preprint by Bonfioli [27] uses a related idea, so we will read it and compare.

2. **Hard sets.** Test the Question above for larger n.

3. Meet with Dr. Biniaz to review these results, and continue the seminar reports for the course.

Winter 2027 (COMP-4960B).

4. **Why some n are easy.** Look at the lonely times the search uses, and try to explain them. For example, when $2n+1$ is prime, times with denominator $2n+1$ seem to handle almost everything. The goal is a short proof for whole families of n.

5. **How large the denominators must be.** Measure the smallest denominator of a lonely time that every speed set needs, and compare it with the times used by Bohman and Peng [7].

6. **Tight sets.** List the tight sets for larger n and compare them with the pattern of Goddyn and Wong [25].

7. Write the final report and prepare the presentation.

References

[1] Matthieu Rosenfeld, *The lonely runner conjecture holds for eight runners*, arXiv:2509.14111, 2025.

[2] Touch Sungkawichai and Tanupat Trakulthongchai, *Eleven, twelve, and thirteen lonely runners*, arXiv:2604.23906, 2026.

[3] Ram Krishna Pandey, *A Note on the Lonely Runner Conjecture*, Journal of Integer Sequences, Vol. 12, Article 09.4.6, 2009.

[4] Tanupat Trakulthongchai, *Nine and ten lonely runners*, arXiv:2511.22427, 2025.

[5] Romanos Diogenes Malikiosis, Francisco Santos, and Matthias Schymura, *Linearly-exponential checking is enough for the lonely runner conjecture and some of its variants*, Forum of Mathematics, Sigma, Vol. 13, e164, 2025.

[6] Terence Tao, *Some remarks on the lonely runner conjecture*, Contributions to Discrete Mathematics, Vol. 13, No. 2, pp. 1–31, 2018.

[7] Tom Bohman and Fei Peng, *Coprime Mappings and Lonely Runners*, arXiv:2109.09860, 2021.

[8] Jörg M. Wills, *Two Problems in Metric Diophantine Approximation*, in *Number Theory and Algebra*, Academic Press, 1977.

[9] T. W. Cusick, *View-Obstruction Problems*, Aequationes Mathematicae, Vol. 9, pp. 165–170, 1973.

[10] Tom Bohman, Ron Holzman, and Daniel J. Kleitman, *Six Lonely Runners*, The Electronic Journal of Combinatorics, Vol. 8, No. 2, Research Paper 3, 2001.

[11] Jiri Cibulka, *On the Lonely Runner Conjecture for Nearly All Runners*, Mathematica Bohemica, Vol. 135, No. 1, 2010.

[12] Arturas Dubickas, *The Lonely Runner Problem for Many Runners*, Glasnik Matematicki, Vol. 46(66), No. 1, 2011.

[13] Guillem Perarnau and Oriol Serra, *The Lonely Runner Conjecture turns 60*, Computer Science Review, Vol. 58, Article 100798, 2025.

[14] Jörg M. Wills, *Zwei Sätze über inhomogene diophantische Approximation von Irrationalzahlen*, Monatshefte für Mathematik, Vol. 71, No. 3, pp. 263–269, 1967.

[15] N. M. Haralambis, *Sets of integers with missing differences*, Journal of Combinatorial Theory, Series A, Vol. 23, No. 1, pp. 22–33, 1977.

[16] Yong-Gao Chen, *View-obstruction problems in n-dimensional Euclidean space and a generalization of them*, Acta Mathematica Sinica, 1994.

[17] Yong-Gao Chen and T. W. Cusick, *The view-obstruction problem for n-dimensional cubes*, Journal of Number Theory, Vol. 74, No. 1, pp. 126–133, 1999.

[18] Ram Krishna Pandey, *On the lonely runner conjecture*, Mathematica Bohemica, Vol. 135, No. 1, pp. 63–68, 2010.

[19] T. W. Cusick and Carl Pomerance, *View-obstruction problems, III*, Journal of Number Theory, Vol. 19, No. 2, pp. 131–139, 1984.

[20] Jérôme Renault, *View-obstruction: a shorter proof for 6 lonely runners*, Discrete Mathematics, Vol. 287, No. 1, pp. 93–101, 2004.

[21] Carl Pomerance, *Coprime matchings of intervals*, 2021.

[22] Ulrich Betke and Jörg M. Wills, *Untere Schranken für zwei diophantische Approximations-Funktionen*, Monatshefte für Mathematik, Vol. 76, No. 3, pp. 214–217, 1972.

[23] Javier Barajas and Oriol Serra, *The lonely runner with seven runners*, The Electronic Journal of Combinatorics, Vol. 15, No. 1, Research Paper R48, 2008.

[24] Benjamin Bedert, *Riesz products and the lonely runner conjecture: a wider gap of loneliness*, arXiv:2511.16636, 2025.

[25] Luis Goddyn and Erick B. Wong, *Tight instances of the lonely runner*, INTEGERS: Electronic Journal of Combinatorial Number Theory, Vol. 6, 2006.

[26] Jaan Allikvere, *Fourteen and fifteen lonely runners*, arXiv:2609.02604, 2026. Preprint.

[27] Vico Bonfioli, *Lonely runner conjecture: dense windows and the covering framework, an obstruction catalogue, a perturbation bound, a reduction, and eight runners*, version 1.0.3 (draft, not peer reviewed), October 2026. `https://github.com/ElVec1o/lonely-runner-covering`.

[28] Asfi Hassan, *Summer 2026 USRA survey paper on the Lonely Runner Conjecture*, University of Windsor, 2026. `https://asfihassan.github.io/Portfolio/Survey_Paper_USRA.pdf`.

[29] Asfi Hassan, *lonely-runner-geometric-search* (software repository), 2026. `https://github.com/asfihassan/lonely-runner-geometric-search`.